

Controladores PID

Simulación en MATLAB y Simulink

Prof. D.Sc. BARSEKH-ONJI Aboud

Facultad de Ingeniería
Universidad Anáhuac México

17 de marzo de 2026

Tabla de Contenido

- 1 Recapitulación: Sistemas Dinámicos
- 2 Teoría del Controlador PID
- 3 Modelado en Simulink
- 4 Sintonización del PID
- 5 Análisis de Respuesta en MATLAB
- 6 Interpretación de Resultados
- 7 Conclusiones

Agenda

- 1 Recapitulación: Sistemas Dinámicos
- 2 Teoría del Controlador PID
- 3 Modelado en Simulink
- 4 Sintonización del PID
- 5 Análisis de Respuesta en MATLAB
- 6 Interpretación de Resultados
- 7 Conclusiones

¿Dónde estamos?

Continuidad de la materia

En sesiones anteriores aprendimos a **modelar** y **simular** sistemas dinámicos:

Lo que ya sabemos:

- Obtener la ecuación diferencial de un sistema físico
- Aplicar la Transformada de Laplace para obtener $G(s)$
- Simular la respuesta en Simulink con bloques de *Transfer Function*
- Interpretar la respuesta temporal (transitorio y estado estacionario)

Lo que aprenderemos hoy:

- Por qué el sistema abierto no es suficiente
- Qué es un controlador PID y cómo actúa
- Cómo cerrar el lazo en Simulink
- Cómo sintonizar el PID con herramientas de MATLAB
- Cómo analizar y comparar respuestas

El Problema: Sistema en Lazo Abierto

Limitación fundamental del lazo abierto

Un sistema en lazo abierto aplica su entrada sin verificar si la salida es la correcta. Cualquier perturbación o error en el modelo genera una respuesta incorrecta **sin posibilidad de corrección automática**.

Consideremos nuestra planta ya conocida:

$$G(s) = \frac{1}{s^2 + 2s + 4} \quad (1)$$

- Si aplicamos una entrada escalón $R(s) = \frac{1}{s}$, la salida converge a $y(\infty) = \frac{1}{4} = 0,25$ (por el teorema del valor final)
- Pero si la referencia deseada es $r = 1,0$, **el sistema en lazo abierto nunca la alcanza**
- Necesitamos **retroalimentación (feedback)** para corregir este error

De Lazo Abierto a Lazo Cerrado

Lazo Abierto:

$$R(s) \rightarrow G(s) \rightarrow Y(s)$$

- No hay corrección automática
- Sensible a perturbaciones
- Error de estado estacionario persistente

Lazo Cerrado (con controlador):

$$E(s) = R(s) - Y(s)$$

$$U(s) = C(s) \cdot E(s)$$

$$Y(s) = G(s) \cdot U(s)$$

- El error se mide y corrige continuamente
- Robusto frente a perturbaciones
- El controlador $C(s)$ determina la calidad de la respuesta

Función de transferencia en lazo cerrado

$$T(s) = \frac{C(s) \cdot G(s)}{1 + C(s) \cdot G(s)}$$

Agenda

- 1 Recapitulación: Sistemas Dinámicos
- 2 Teoría del Controlador PID
- 3 Modelado en Simulink
- 4 Sintonización del PID
- 5 Análisis de Respuesta en MATLAB
- 6 Interpretación de Resultados
- 7 Conclusiones

¿Qué es un Controlador PID?

Definición

Un controlador **PID** (Proporcional–Integral–Derivativo) calcula una señal de control $u(t)$ a partir del **error** $e(t) = r(t) - y(t)$, combinando tres acciones correctivas independientes.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d e(t)}{dt} \quad (2)$$

- K_p — Ganancia **Proporcional**
- K_i — Ganancia **Integral**
- K_d — Ganancia **Derivativa**
- $e(t) = r(t) - y(t)$ — Error entre referencia y salida actual

Acción Proporcional (P)

Principio

La acción proporcional aplica una corrección **directamente proporcional** al error actual.

$$u_P(t) = K_p \cdot e(t) \quad (3)$$

Efecto de aumentar K_p :

- $\uparrow$ Velocidad de respuesta
- $\downarrow$ Tiempo de subida (t_r)
- $\uparrow$ Sobreimpulso (*overshoot*)
- Puede generar inestabilidad

Limitación clave

La acción P sola **no elimina el error de estado estacionario** ($e_{ss} \neq 0$) en sistemas tipo 0. Siempre queda un error residual proporcional a $\frac{1}{1+K_p \cdot K_{DC}}$.

Acción Integral (I)

Principio

La acción integral acumula el error en el tiempo. Si hay error persistente, la acción integral crece hasta que el error se elimine.

$$u_I(t) = K_i \int_0^t e(\tau) d\tau \quad (4)$$

Efecto de aumentar K_i :

- Elimina el **error de estado estacionario**
- ↑ Sobreimpulso
- ↓ Estabilidad (puede oscilar)
- Respuesta más lenta al inicio

Integrator Windup

Si la actuación está saturada y el error persiste, la integral sigue creciendo (*windup*). Esto puede causar sobreimpulsos grandes al salir de la saturación. Los PID modernos incluyen **anti-windup**.

Acción Derivativa (D)

Principio

La acción derivativa reacciona a la **velocidad de cambio** del error. Predice el error futuro y aplica una corrección anticipada.

$$u_D(t) = K_d \frac{d e(t)}{dt} \quad (5)$$

Efecto de aumentar K_d :

- ↓ Sobreimpulso
- Amortigua oscilaciones
- ↑ Estabilidad (hasta cierto punto)
- Mejora la respuesta transitoria

La derivada amplifica el **ruido de medición**. En la práctica, se limita su banda de acción con un filtro de primer orden:

$$C_D(s) = \frac{K_d s}{1 + s/N}$$

donde N es el coeficiente de filtro (típicamente $5 < N < 20$)

Función de Transferencia del PID

En el dominio de Laplace, el controlador PID tiene la siguiente función de transferencia:

$$C(s) = K_p + \frac{K_i}{s} + K_d s = \frac{K_d s^2 + K_p s + K_i}{s} \quad (6)$$

Forma estándar (ISA):

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s \right) \quad (7)$$

- $T_i = K_p / K_i$ — Tiempo integral (reset time)
- $T_d = K_d / K_p$ — Tiempo derivativo

Función de lazo cerrado completa

$$T(s) = \frac{C(s) \cdot G(s)}{1 + C(s) \cdot G(s)} = \frac{(K_d s^2 + K_p s + K_i)}{s(s^2 + 2s + 4) + (K_d s^2 + K_p s + K_i)}$$

Resumen: Efecto de las Ganancias PID

Parámetro	t_r (subida)	Sobreimpulso	t_s (asentam.)	e_{ss}	Estabilidad
$\uparrow K_p$	Disminuye	Aumenta	Pequeño cambio	Disminuye	Empeora
$\uparrow K_i$	Disminuye	Aumenta	Aumenta	Elimina	Empeora
$\uparrow K_d$	Pequeño cambio	Disminuye	Disminuye	Sin cambio	Mejora

Regla práctica

El ajuste de las ganancias PID es un balance entre rapidez, sobreimpulso y estabilidad. No existe un ajuste perfecto universal: depende de los **requerimientos del sistema** y del **proceso físico**.

Agenda

- 1 Recapitulación: Sistemas Dinámicos
- 2 Teoría del Controlador PID
- 3 Modelado en Simulink**
- 4 Sintonización del PID
- 5 Análisis de Respuesta en MATLAB
- 6 Interpretación de Resultados
- 7 Conclusiones

Arquitectura del Sistema de Control en Simulink

Diagrama de bloques a implementar

El sistema de control en lazo cerrado con controlador PID se construye conectando los siguientes bloques en Simulink:

Bloque	Librería Simulink	Función
Step	Sources	Señal de referencia $r(t)$
Sum	Math Operations	Cálculo del error $e = r - y$
Gain	Math Operations	Ganancia de cadena directa
PID Controller	Continuous	Controlador PID
Transfer Fcn	Continuous	Planta $G(s)$
Scope	Sinks	Visualización de señales

Configuración del Bloque Transfer Fcn (Planta)

Planta del sistema

La función de transferencia de la planta que usamos como ejemplo es:

$$G(s) = \frac{1}{s^2 + 2s + 4} \quad (8)$$

Análisis de la planta:

- Sistema de **segundo orden** subamortiguado ($\zeta < 1$)
- Polos en $s = -1 \pm j\sqrt{3}$ (p.real (-) $\Rightarrow$ estable)
- Ganancia DC: $G(0) = 1/4 = 0,25$
- Sin ceros
- En lazo abierto, ante escalón unitario:
 $y(\infty) = 0,25$

Configuración en Simulink:

- Numerador: [1]
- Denominador: [1 2 4]
- Doble clic en el bloque y se ingresan estos vectores de coeficientes (de mayor a menor potencia de s)

Configuración del Bloque Sum y Gain

Bloque Sum (Nodo de error):

- Doble clic → **List of signs**: +-
 - La primera entrada (+) recibe la referencia r
 - La segunda entrada (-) recibe la retroalimentación y
- Salida: $e(t) = r(t) - y(t)$

Atención

Verificar que el signo de retroalimentación sea negativo (-). Un error aquí convierte el lazo en positivo, desestabilizando el sistema.

Bloque Gain (Ganancia de cadena directa):

- Valor: 2.5 (escala el error antes del PID)
- Equivale a un ajuste adicional de amplificación en la cadena directa
- Se puede interpretar como parte del controlador proporcional total

Nota pedagógica

Este bloque Gain nos permite observar cómo una ganancia en la cadena directa modifica la respuesta, **independientemente** de las ganancias PID.

Configuración del Bloque PID Controller

Parámetros del bloque PID en Simulink

El bloque PID Controller (librería *Continuous*) implementa la forma paralela o estándar del PID.

Parámetros principales:

- **Controller:** PID / PI / PD / P
- **Form:** Parallel o Ideal
- **P:** ganancia proporcional K_p
- **I:** ganancia integral K_i
- **D:** ganancia derivativa K_d
- **N:** coeficiente del filtro derivativo

Opciones avanzadas:

- **Anti-windup:** limita la acumulación de la integral durante saturación
- **Output saturation:** define los límites físicos del actuador
- **Enable tracking mode:** para conmutación entre controladores

Configuración del Bloque PID Controller

Punto de partida

Para la primera simulación: $K_p = 1$, $K_i = 0$, $K_d = 0$ (solo proporcional). Luego sintonizaremos.

Procedimiento para Construir el Modelo

- 1 Abrir MATLAB y escribir `simulink` en la línea de comandos
- 2 Crear un nuevo modelo en blanco: `File` → `New` → `Model`
- 3 Agregar los bloques desde el *Library Browser* (buscar por nombre)
- 4 Conectar los bloques en el orden: `Step` → `Sum` → `Gain` → `PID` → `Transfer Fcn` → `Scope`
- 5 Llevar la salida de `Transfer Fcn` de regreso a la entrada negativa del `Sum`
- 6 Conectar tanto $r(t)$ como $y(t)$ al `Scope` (configurarlo con 2 entradas)
- 7 Configurar los parámetros de simulación: `Modeling` → `Model Settings`
 - Tiempo de parada: 30 segundos
 - Solver: `ode45` (apropiado para sistemas continuos)
- 8 Ejecutar la simulación con `Run` (▶) y observar la respuesta en el `Scope`

Agenda

- 1 Recapitulación: Sistemas Dinámicos
- 2 Teoría del Controlador PID
- 3 Modelado en Simulink
- 4 Sintonización del PID
- 5 Análisis de Respuesta en MATLAB
- 6 Interpretación de Resultados
- 7 Conclusiones

¿Por Qué Sintonizar el PID?

Problema de la sintonización

Elegir valores incorrectos para K_p , K_i , K_d puede resultar en respuestas inaceptables o en un sistema inestable.

Criterios de desempeño típicos:

- **Tiempo de subida** t_r : rapidez de la respuesta
- **Sobreimpulso** M_p : máximo porcentaje sobre la referencia
- **Tiempo de asentamiento** t_s : cuándo la respuesta se queda en $\pm 2\%$ o $\pm 5\%$ de la referencia
- **Error de estado estacionario** e_{ss} : error residual permanente

Métodos de sintonización:

- **Ziegler-Nichols** (manual, clásico)
- **PID Tuner App** (automático, MATLAB)
- **Optimización** (basada en criterios ITAE, ISE, IAE)
- **Prueba y error** (válido para aprendizaje)

Método de Ziegler-Nichols (Referencia Teórica)

Método de la ganancia última (lazo cerrado)

Este método clásico determina parámetros PID a partir de la **ganancia última** K_u y el **período de oscilación sostenida** T_u .

Procedimiento:

- 1 Desactivar las acciones I y D ($K_i = 0$, $K_d = 0$)
- 2 Aumentar K_p hasta que el sistema oscile de forma **sostenida y constante**
- 3 Registrar K_u (ganancia en ese punto) y T_u (período de oscilación en segundos)
- 4 Aplicar las fórmulas de la tabla:

Tipo	K_p	T_i	T_d
P	$0,50 K_u$	—	—
PI	$0,45 K_u$	$T_u/1,2$	—
PID	$0,60 K_u$	$T_u/2$	$T_u/8$

La App *PID Tuner* de MATLAB/Simulink

¿Qué es el PID Tuner?

Es una herramienta interactiva integrada en Simulink que **lineariza automáticamente** el modelo y propone ganancias óptimas mediante técnicas de diseño en frecuencia.

Cómo acceder al PID Tuner:

- 1 Doble clic sobre el bloque PID Controller en el modelo
- 2 En la ventana del bloque, hacer clic en “**Tune...**” (o “Manage and Tune” según versión)
- 3 Simulink linealiza el modelo alrededor del punto de operación actual
- 4 Se abre la ventana del **PID Tuner**

Ventaja

No requiere conocer la planta analíticamente. El PID Tuner la identifica desde el modelo de Simulink automáticamente.

Uso del PID Tuner: Controles de la Interfaz

Elementos principales de la interfaz:

- **Response plot:** muestra la respuesta al escalón con las ganancias actuales
- **Performance/Robustness slider:** desplazar hacia la izquierda para más robustez, hacia la derecha para más rapidez
- **Bandwidth / Phase margin:** métricas en frecuencia para la sintonización
- **Controller parameters:** muestra los valores de K_p , K_i , K_d calculados

Flujo de trabajo:

- 1 Observar la respuesta inicial propuesta
- 2 Ajustar el slider para balancear rapidez vs. robustez
- 3 Verificar el sobreimpulso y el tiempo de asentamiento
- 4 Verificar el margen de fase (ideal > 60)
- 5 Hacer clic en **“Update Block”** para aplicar al modelo
- 6 Ejecutar la simulación y comparar

Agenda

- 1 Recapitulación: Sistemas Dinámicos
- 2 Teoría del Controlador PID
- 3 Modelado en Simulink
- 4 Sintonización del PID
- 5 Análisis de Respuesta en MATLAB**
- 6 Interpretación de Resultados
- 7 Conclusiones

Simulación desde MATLAB: Definir el Sistema

Enfoque alternativo: usar MATLAB directamente (sin Simulink)

MATLAB permite simular y analizar el mismo sistema de control mediante comandos, lo cual es útil para comparar múltiples sintonías rápidamente.

```
1 % =====
2 % Definicion del sistema de control PID en lazo cerrado
3 % Materia: Simulacion | UAM - Ingenieria
4 % =====
5 % Planta  $G(s) = 1 / (s^2 + 2s + 4)$ 
6 num_G = [1];
7 den_G = [1 2 4];
8 G = tf(num_G, den_G);
9
10 % Controlador PID:  $C(s) = K_p + K_i/s + K_d*s$ 
11 Kp = 1.0;    Ki = 0.0;    Kd = 0.0;    % Solo proporcional (inicio)
12
```



Simulación: Respuesta al Escalón y Análisis

```
1 % =====
2 % Comparacion de distintas sintonias PID
3 % =====
4 t = 0:0.01:30;    % Vector de tiempo (0 a 30 segundos)
5
6 % Solo Proporcional (P)
7 C_P = pid(1.0, 0, 0);    T_P = feedback(C_P * G, 1);
8 % Proporcional + Integral (PI)
9 C_PI = pid(1.5, 0.8, 0);    T_PI = feedback(C_PI * G, 1);
10 % PID completo (sintonizado)
11 C_PID = pid(2.0, 1.2, 0.5); T_PID = feedback(C_PID * G, 1);
12
13 figure;
14 step(T_P, T_PI, T_PID, t);
15 legend('Solo_P', 'PI', 'PID');
16 title('Comparacion de respuestas: P vs PI vs PID');
17 xlabel('Tiempo (s)'); ylabel('Salida y(t)');
18 grid on;
```



Sintonización Automática con pidtune

Comando pidtune

MATLAB incluye la función `pidtune()` que diseña automáticamente un controlador PID dado el modelo de la planta y un tipo de controlador.

```
1 % =====  
2 % Sintonia automatica con pidtune()  
3 % =====  
4 % Planta  
5 G = tf(1, [1 2 4]);  
6  
7 % Sintonizar un controlador PID con pidtune  
8 [C_auto, info] = pidtune(G, 'PID');  
9  
10 fprintf('---_Ganancias_sintonizadas_automaticamente_---\n');  
11 fprintf('Kp=_%.4f\n', C_auto.Kp);  
12 fprintf('Ki=_%.4f\n', C_auto.Ki);
```



Análisis Completo: stepinfo y Comparativa

```
1 % =====
2 % Analisis de desempeno con stepinfo()
3 % =====
4 % Sistemas a comparar
5 G = tf(1, [1 2 4]);
6 C1 = pid(1, 0, 0); T1 = feedback(C1 * G, 1); % P
7 C2 = pid(1.5, 0.8, 0); T2 = feedback(C2 * G, 1); % PI
8 C3 = pid(2, 1.2, 0.5); T3 = feedback(C3 * G, 1); % PID
9
10 S1 = stepinfo(T1); S2 = stepinfo(T2); S3 = stepinfo(T3);
11
12 fprintf('\n%-20s_ %8s_ %8s_ %8s\n', 'Metrica', 'P', 'PI', 'PID');
13 fprintf(' %-20s_ %8.3f_ %8.3f_ %8.3f\n', 't_rise_(s)', ...
14     S1.RiseTime, S2.RiseTime, S3.RiseTime);
15 fprintf(' %-20s_ %8.2f_ %8.2f_ %8.2f\n', 'Overshoot_(%)', ...
16     S1.Overshoot, S2.Overshoot, S3.Overshoot);
17 fprintf(' %-20s_ %8.3f_ %8.3f_ %8.3f\n', 't_settle_(s)', ...
18     S1.SettlingTime, S2.SettlingTime, S3.SettlingTime);
```



Agenda

- 1 Recapitulación: Sistemas Dinámicos
- 2 Teoría del Controlador PID
- 3 Modelado en Simulink
- 4 Sintonización del PID
- 5 Análisis de Respuesta en MATLAB
- 6 Interpretación de Resultados**
- 7 Conclusiones

Métricas de Desempeño: ¿Qué Observar en el Scope?

En la respuesta al escalón, analizar:

- **Tiempo de retardo** t_d : tiempo en que la salida alcanza el 50 % del valor final
- **Tiempo de subida** t_r : tiempo de 10 % a 90 % del valor final
- **Tiempo de pico** t_p : instante del máximo sobreimpulso
- **Sobreimpulso** M_p : $\frac{y_{max} - y_{ss}}{y_{ss}} \times 100 \%$
- **Tiempo de asentamiento** t_s : cuando la respuesta entra y permanece en banda $\pm 2 \%$

Criterios aceptables (típicos en ingeniería):

- $M_p < 5 \%$ (aplicaciones críticas)
- $M_p < 20 \%$ (aplicaciones generales)
- $e_{ss} = 0$ (con acción integral)
- t_s mínimo según requisitos

Regla general

Un buen diseño de control siempre especifica primero los **requerimientos de desempeño** antes de elegir las ganancias.

Comparativa de Respuestas: Efecto de la Sintonía

Lectura de la tabla de resultados esperados (ejemplo)

Al ejecutar el código de comparación, se obtienen resultados similares a los siguientes para $G(s) = 1/(s^2 + 2s + 4)$:

Controlador	t_r (s)	M_p (%)	t_s (s)	e_{ss}
Solo P ($K_p = 1$)	$\sim 2,5$	0	~ 5	0,20
PI ($K_p = 1,5$, $K_i = 0,8$)	$\sim 1,8$	~ 8	~ 12	0
PID ($K_p = 2$, $K_i = 1,2$, $K_d = 0,5$)	$\sim 1,2$	~ 5	~ 8	0
pidtune() automático	Óptimo	Mínimo	Mínimo	0

Observación pedagógica

La acción P sola reduce el tiempo de subida pero deja error residual. La acción I lo elimina a costo de mayor sobreimpulso. La acción D reduce el sobreimpulso y el tiempo de asentamiento.

Diagrama de Bode y Margen de Estabilidad

```
1 % =====
2 % Analisis de estabilidad: Diagrama de Bode y margenes
3 % =====
4 G = tf(1, [1 2 4]);
5 C = pid(2, 1.2, 0.5);
6 L = C * G;          % Funcion de lazo abierto L(s)
7
8 % Margenes de ganancia y fase
9 [Gm, Pm, Wcg, Wcp] = margin(L);
10 fprintf('Margen de ganancia: %.2f dB (%.2f veces)\n', 20*log10(Gm), Gm);
11 fprintf('Margen de fase: %.2f grados\n', Pm);
12 fprintf('Freq. cruce ganancia: %.4f rad/s\n', Wcg);
13 fprintf('Freq. cruce fase: %.4f rad/s\n', Wcp);
14
15 % Graficar Diagrama de Bode del lazo abierto
16 figure;
17 margin(L);          % margin() grafica el Bode y marca los margenes
18 title('Diagrama de Bode - Lazo Abierto con PID');
```



Agenda

- 1 Recapitulación: Sistemas Dinámicos
- 2 Teoría del Controlador PID
- 3 Modelado en Simulink
- 4 Sintonización del PID
- 5 Análisis de Respuesta en MATLAB
- 6 Interpretación de Resultados
- 7 Conclusiones

Conclusiones

Lo que aprendimos en esta sesión

- 1 Los sistemas en lazo abierto tienen error de estado estacionario y no corrigen perturbaciones automáticamente.
- 2 El controlador PID combina tres acciones: **P** (rapidez), **I** (precisión), **D** (amortiguación).
- 3 En Simulink, el lazo cerrado se construye conectando: Step $\rightarrow$ Sum $\rightarrow$ PID $\rightarrow$ Plant $\rightarrow$ (retroalimentación).
- 4 La **App PID Tuner** y el comando `piddtune()` permiten sintonizar automáticamente.
- 5 El comando `stepinfo()` cuantifica el desempeño y permite comparar sintonías.
- 6 El **Diagrama de Bode** y los márgenes de fase/ganancia verifican la estabilidad del diseño.

Conclusiones

Para la práctica

Experimentar con diferentes valores de K_p , K_i , K_d y documentar el efecto en t_r , M_p , t_s y e_{ss} mediante una tabla comparativa.

Referencias y Recursos

Documentación oficial MATLAB/Simulink

<https://la.mathworks.com/help/simulink/>

<https://la.mathworks.com/help/simulink/slref/pidcontroller.html>

<https://la.mathworks.com/help/control/ref/pidtune.html>

Bibliografía de referencia

- Ogata, K. (2010). *Modern Control Engineering* (5th ed.). Prentice Hall.
- Dorf, R. & Bishop, R. (2017). *Modern Control Systems* (13th ed.). Pearson.
- Franklin, G., Powell, J. & Emami-Naeini, A. (2019). *Feedback Control of Dynamic Systems*. Pearson.